PART 2: BAYESIAN PROBABILITY - IMDB MOVIE ANALYSIS

Formative 3 Assignment - Bayesian Probability Implementation

This implementation uses Bayes' Theorem to compute posterior probabilities P(Positive | keyword) for selected sentiment keywords from IMDb reviews.

BAYES' THEOREM FORMULA:

P(Positive | keyword) = P(keyword | Positive) × P(Positive) / P(keyword)

Where: P(Positive | keyword) = POSTERIOR (what we want to find) P(keyword | Positive) = LIKELIHOOD (how common in positive reviews) P(Positive) = PRIOR (baseline probability) P(keyword) = MARGINAL (how common overall)

REQUIREMENTS MET:

- Pure Python only (csv, re modules only)
- 3 positive + 3 negative keywords
- All 4 probabilities calculated: Prior, Likelihood, Marginal, Posterior
- Bayes' Theorem correctly implemented: P(Pos|kw) = P(kw|Pos) × P(Pos) / P(kw)
- Probability tables in professional format
- Code demonstrates Python skills (modular, DRY principle)

Author: Rolande

Assignment: Formative 3 - Part 2

Date: 02/07/2026

```
import csv
import re

DATASET_PATH = "IMDB Dataset.csv"

POSITIVE_KEYWORDS = ["excellent", "wonderful", "brilliant"]
NEGATIVE_KEYWORDS = ["awful", "horrible", "boring"]
ALL_KEYWORDS = POSITIVE_KEYWORDS + NEGATIVE_KEYWORDS


# ============================================================
# STEP 1: DATA LOADING
# ============================================================

def load_reviews(filepath):
    """Yield (cleaned_review_text, sentiment) for every row in the CSV."""
    processed_count = 0
    skipped_count = 0
    skipped_rows_info = []

    with open(filepath, "r", encoding="utf-8") as file:
        reader = csv.DictReader(file)
        for i, row in enumerate(reader):
            current_skipped_reasons = []

            # Check for 'review' key presence and value being None
            if 'review' not in row:
                current_skipped_reasons.append("'review' key missing")
            elif row['review'] is None:
                current_skipped_reasons.append("'review' value is None")

            # Check for 'sentiment' key presence and value being None
            if 'sentiment' not in row:
```

```
            if 'sentiment' not in row:
                current_skipped_reasons.append("'sentiment' key missing")
            elif row['sentiment'] is None:
                current_skipped_reasons.append("'sentiment' value is None")

            if current_skipped_reasons:
                skipped_count += 1
                skipped_rows_info.append((i + 2, current_skipped_reasons, row))
                continue

            # If not skipped by initial checks, proceed to process
            text = row["review"].lower()
            text = re.sub(r"<br\s*/?>", " ", text)   # strip HTML line breaks
            text = re.sub(r"<[^>]+>", "", text)       # strip any other HTML tags
            sentiment = row["sentiment"].strip().lower()

            # Also check if sentiment value is valid ('positive' or 'negative')
            if sentiment not in ["positive", "negative"]:
                skipped_count += 1
                skipped_rows_info.append((i + 2, [f"Invalid sentiment value: '{sentiment}'"], row))
                continue

            processed_count += 1
            yield text, sentiment

    # Store the skipped rows information in a global variable for later inspection
    global _skipped_review_rows_diagnostic
    _skipped_review_rows_diagnostic = {
        "processed_count": processed_count,
        "skipped_count": skipped_count,
        "skipped_rows_data": skipped_rows_info
    }
    print(f"DEBUG: Total reviews processed by load_reviews: {processed_count}")
    print(f"DEBUG: Total reviews skipped by load_reviews: {skipped_count}")
    if skipped_rows_info:
        print(f"DEBUG: First 5 skipped rows (line_num, reasons, data): {skipped_rows_info[:5]}")


def count_reviews_by_sentiment(filepath):
    """Return (total_reviews, positive_count, negative_count)."""
    total = positive = negative = 0
    for _, sentiment in load_reviews(filepath):
        total += 1
        if sentiment == "positive":
            positive += 1
        elif sentiment == "negative":
            negative += 1
    return total, positive, negative


# ============================================================
# STEP 2: KEYWORD COUNTING
# ============================================================

def compile_keyword_patterns(keywords):
    """Whole-word regex patterns so 'excellent' doesn't match 'excellently'."""
    return {kw: re.compile(r"\b" + re.escape(kw) + r"\b") for kw in keywords}


def count_keyword_occurrences(filepath, keywords):
    """
    For each keyword, count how many positive and how many negative
    reviews contain it (presence-based: one match per review, not per word).
    """
    patterns = compile_keyword_patterns(keywords)
    counts = {kw: {"positive": 0, "negative": 0} for kw in keywords}

    for text, sentiment in load_reviews(filepath):
        for keyword, pattern in patterns.items():
            if pattern.search(text):
                if sentiment == "positive":
                    counts[keyword]["positive"] += 1
```

```
                elif sentiment == "negative":
                    counts[keyword]["negative"] += 1
    return counts


# ============================================================
# STEP 3: BAYES' THEOREM
# ============================================================

def calculate_bayes_probabilities(keyword_pos_count, keyword_neg_count,
                                  total_reviews, total_positive_reviews):
    """
    Compute Prior, Likelihood, Marginal, and Posterior for ONE keyword.
    Posterior is always P(Positive | keyword) -- never P(Negative | keyword).
    """
    keyword_total = keyword_pos_count + keyword_neg_count

    prior = total_positive_reviews / total_reviews                          # P(Positive)
    likelihood = (keyword_pos_count / total_positive_reviews
                  if total_positive_reviews > 0 else 0)                     # P(keyword|Positive)
    marginal = keyword_total / total_reviews                                # P(keyword)
    posterior = (likelihood * prior / marginal) if marginal > 0 else 0.0    # P(Positive|keyword)

    return {
        "prior": prior,
        "likelihood": likelihood,
        "marginal": marginal,
        "posterior": posterior,
        "keyword_pos": keyword_pos_count,
        "keyword_neg": keyword_neg_count,
        "keyword_total": keyword_total,
    }


def run_bayesian_analysis(filepath, keywords):
    total, positive, negative = count_reviews_by_sentiment(filepath)
    keyword_counts = count_keyword_occurrences(filepath, keywords)

    results = {}
    for kw in keywords:
        c = keyword_counts[kw]
        results[kw] = calculate_bayes_probabilities(
            c["positive"], c["negative"], total, positive
        )

    stats = {"total": total, "positive": positive, "negative": negative}
    return stats, results


# ============================================================
# STEP 4: DISPLAY / PRESENTATION
# ============================================================

def print_dataset_summary(stats):
    print("=" * 70)
    print("DATASET STATISTICS")
    print("=" * 70)
    print(f"Total Reviews:      {stats['total']:>10,}")
    print(f"Positive Reviews:   {stats['positive']:>10,}  "
          f"({stats['positive']/stats['total']:.0%})")
    print(f"Negative Reviews:   {stats['negative']:>10,}  "
          f"({stats['negative']/stats['total']:.0%})")
    print(f"Prior P(Positive):  {stats['positive']/stats['total']:>10.4f}")
    print("=" * 70, "\n")


def print_probability_table(results):
    print("=" * 80)
    print("BAYESIAN PROBABILITY TABLE  (all values are P(Positive | keyword))")
    print("=" * 80)
    header = f"{'Keyword':<12}{'Prior':<9}{'Likelihood':<13}{'Marginal':<11}{'Posterior':<11}"
```

```
    print(header)
    print("-" * 80)
    for kw in ALL_KEYWORDS:
        r = results[kw]
        print(f"{kw:<12}{r['prior']:<9.4f}{r['likelihood']:<13.5f}"
              f"{r['marginal']:<11.5f}{r['posterior']:<11.4f}")
    print("=" * 80, "\n")


def print_interpretation(results):
    """
    Correct, non-contradictory interpretation.
    IMPORTANT: the percentage shown is ALWAYS P(Positive|keyword).
    We only change the *label* (positive/negative indicator), never
    silently swap in a different number for the negative case.
    """
    print("=" * 80)
    print("INTERPRETATION")
    print("=" * 80)
    for kw in ALL_KEYWORDS:
        r = results[kw]
        p = r["posterior"] * 100
        distance_from_baseline = abs(r["posterior"] - 0.5)

        if distance_from_baseline > 0.25:
            strength = "VERY STRONG"
        elif distance_from_baseline > 0.10:
            strength = "STRONG"
        elif distance_from_baseline > 0.02:
            strength = "MODERATE"
        else:
            strength = "WEAK"

        print(f"\n'{kw}':")
        print(f"  P(Positive | '{kw}') = {p:.1f}%")
        if r["posterior"] > 0.5:
            print(f"  -> {strength} POSITIVE indicator")
        else:
            print(f"  -> {strength} NEGATIVE indicator "
                  f"(equivalently, {100 - p:.1f}% likely negative)")
    print("\n" + "=" * 80, "\n")


def generate_markdown_tables(results):
    """One markdown block per keyword, matching the rubric's required table."""
    md = "# Bayesian Probability Tables (P(Positive | keyword))\n\n"
    for kw in ALL_KEYWORDS:
        r = results[kw]
        md += f"## Keyword: **{kw}**\n\n"
        md += "| Probability | Formula | Value |\n"
        md += "|---|---|---|\n"
        md += f"| Prior | P(Positive) | {r['prior']:.4f} |\n"
        md += f"| Likelihood | P(keyword\\|Positive) | {r['likelihood']:.5f} |\n"
        md += f"| Marginal | P(keyword) | {r['marginal']:.5f} |\n"
        md += f"| Posterior | P(Positive\\|keyword) | {r['posterior']:.4f} |\n\n"
    return md


# ============================================================
# MAIN
# ============================================================

def main():
    stats, results = run_bayesian_analysis(DATASET_PATH, ALL_KEYWORDS)
    print_dataset_summary(stats)
    print_probability_table(results)
    print_interpretation(results)

    markdown = generate_markdown_tables(results)
    with open("probability_tables.md", "w") as f:
        f.write(markdown)
```

```
    print("Markdown tables saved to 'probability_tables.md'")

    return stats, results


if __name__ == "__main__":
    stats, results = main()
```

```
DEBUG: Total reviews processed by load_reviews: 50000
DEBUG: Total reviews skipped by load_reviews: 0
DEBUG: Total reviews processed by load_reviews: 50000
DEBUG: Total reviews skipped by load_reviews: 0
======================================================================
DATASET STATISTICS
======================================================================
Total Reviews:          50,000
Positive Reviews:       25,000  (50%)
Negative Reviews:       25,000  (50%)
Prior P(Positive):      0.5000
======================================================================

================================================================================
BAYESIAN PROBABILITY TABLE  (all values are P(Positive | keyword))
================================================================================
Keyword     Prior    Likelihood   Marginal   Posterior
--------------------------------------------------------------------------------
excellent   0.5000   0.11472      0.07104    0.8074
wonderful   0.5000   0.09032      0.05560    0.8122
brilliant   0.5000   0.06348      0.04176    0.7601
awful       0.5000   0.01136      0.05766    0.0985
horrible    0.5000   0.01304      0.04202    0.1552
boring      0.5000   0.02364      0.06102    0.1937
================================================================================

================================================================================
INTERPRETATION
================================================================================

'excellent':
  P(Positive | 'excellent') = 80.7%
  -> VERY STRONG POSITIVE indicator

'wonderful':
  P(Positive | 'wonderful') = 81.2%
  -> VERY STRONG POSITIVE indicator

'brilliant':
  P(Positive | 'brilliant') = 76.0%
  -> VERY STRONG POSITIVE indicator

'awful':
  P(Positive | 'awful') = 9.9%
  -> VERY STRONG NEGATIVE indicator (equivalently, 90.1% likely negative)

'horrible':
  P(Positive | 'horrible') = 15.5%
  -> VERY STRONG NEGATIVE indicator (equivalently, 84.5% likely negative)

'boring':
  P(Positive | 'boring') = 19.4%
  -> VERY STRONG NEGATIVE indicator (equivalently, 80.6% likely negative)

================================================================================

Markdown tables saved to 'probability_tables.md'
```

Checking the exact number of rows parsed by `csv.DictReader`

```
import csv

# Point 1: Run len(list(csv.DictReader(open("IMDB Dataset.csv", encoding="utf-8"))))
with open(DATASET_PATH, "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    total_rows_from_reader = len(list(reader))
```

```
print(f"Total rows parsed by csv.DictReader: {total_rows_from_reader}")
```

```
Total rows parsed by csv.DictReader: 50000
```

Checking for malformed 'sentiment' values

```
import csv

def check_malformed_sentiment(filepath):
    malformed_sentiments = []
    with open(filepath, "r", encoding="utf-8") as file:
        reader = csv.DictReader(file)
        for i, row in enumerate(reader):
            if 'sentiment' not in row or row['sentiment'] is None:
                malformed_sentiments.append((i + 2, 'Missing sentiment key or value is None', row))
            else:
                sentiment = str(row['sentiment']).strip().lower()
                if sentiment not in ["positive", "negative"]:
                    malformed_sentiments.append((i + 2, sentiment, row))
    return malformed_sentiments

malformed_rows = check_malformed_sentiment(DATASET_PATH)

if malformed_rows:
    print(f"Found {len(malformed_rows)} rows with malformed sentiment values or missing sentiment:")
    for line_num, sentiment_val, row_data in malformed_rows:
        print(f"  Line {line_num}: Sentiment '{sentiment_val}', Row: {row_data}")
else:
    print("No rows found with malformed or missing 'sentiment' values (other than 'positive' or 'negati
```

```
No rows found with malformed or missing 'sentiment' values (other than 'positive' or 'negative').
```

Checking for missing or None 'review' values

Now, I want toidentify if any rows have missing or `None` values in the 'review' column, which would explain why `load_reviews` yields fewer rows than `csv.DictReader` initially parses. it was an error i got before now i want to make sure i won't get it again

```
import csv

def check_missing_reviews(filepath):
    missing_review_rows = []
    with open(filepath, "r", encoding="utf-8") as file:
        reader = csv.DictReader(file)
        for i, row in enumerate(reader):
            # Check if 'review' key is missing or its value is None
            if 'review' not in row or row['review'] is None:
                missing_review_rows.append((i + 2, row)) # +2 for header and 0-based index
    return missing_review_rows

missing_reviews = check_missing_reviews(DATASET_PATH)

if missing_reviews:
    print(f"Found {len(missing_reviews)} rows with missing or None 'review' values:")
    for line_num, row_data in missing_reviews:
        print(f"  Line {line_num}: Row: {row_data}")
else:
    print("No rows found with missing or None 'review' values.")
```

```
No rows found with missing or None 'review' values.
```